


# SISTEMAS OPERATIVOS

08011442

**Germán Sánchez Torres, I.S., M.Sc., Ph.D.**

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com




# Tuberías (pipes)

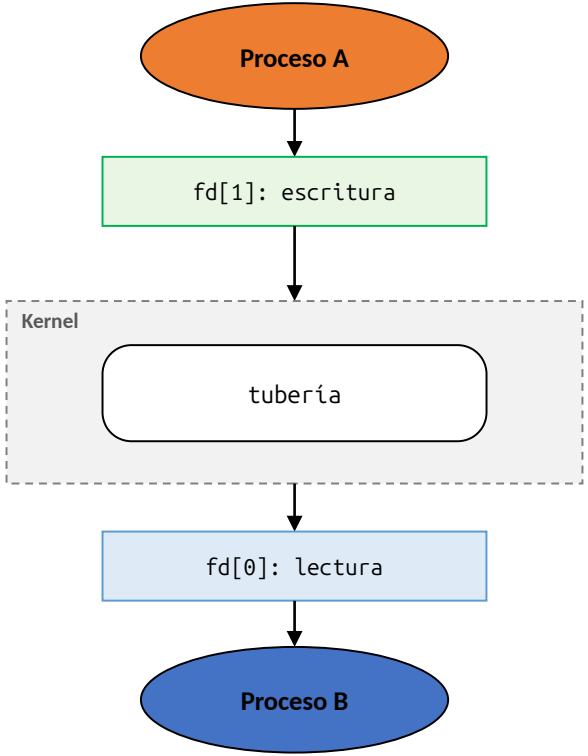
# int pipe(int fd[2])

# Contenido

1. Qué es una tubería
2. Tubería entre dos procesos
3. pipe() + fork() + close(), paso a paso
4. Comunicación bidireccional: dos tuberías
5. Paradigmas de uso
6. Las llamadas pipe, read, write y close
7. Ejemplos
8. Resumen

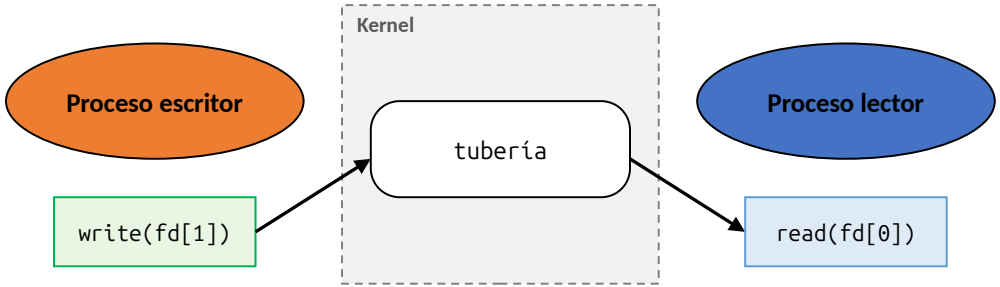
# Características de las tuberías

- Un proceso **escribe** en la tubería como si fuera un archivo y otro proceso **lee** de ella.
- Los datos entran por un extremo y salen por el otro, en el mismo orden (FIFO).
- Es **unidireccional**: para responder hace falta otra tubería.
- Los datos viven en el **kernel** (sistema de archivos pipefs), con capacidad limitada: históricamente 10 bloques de 512 bytes; en Linux, 64 KiB.
- Existe mientras algún proceso tenga abierto alguno de sus dos descriptores.
- Solo comunica procesos **emparentados**: los descriptores se heredan con `fork`.

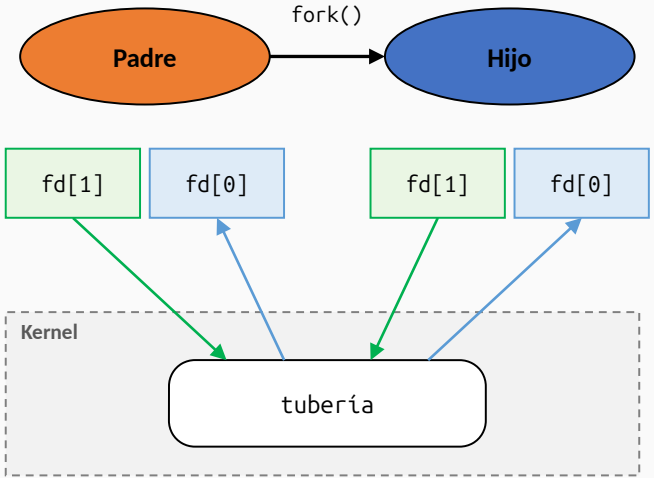


# Tubería entre dos procesos

- `pipe(fd)` devuelve un arreglo de **dos descriptores de archivo**: `fd[0]` es el extremo de lectura y `fd[1]` el de escritura.
- El proceso escritor usa `write(fd[1], …)`; el lector usa `read(fd[0], …)`.
- Tras `fork()` el hijo hereda **copias** de ambos descriptores: los dos procesos apuntan a la misma tubería.



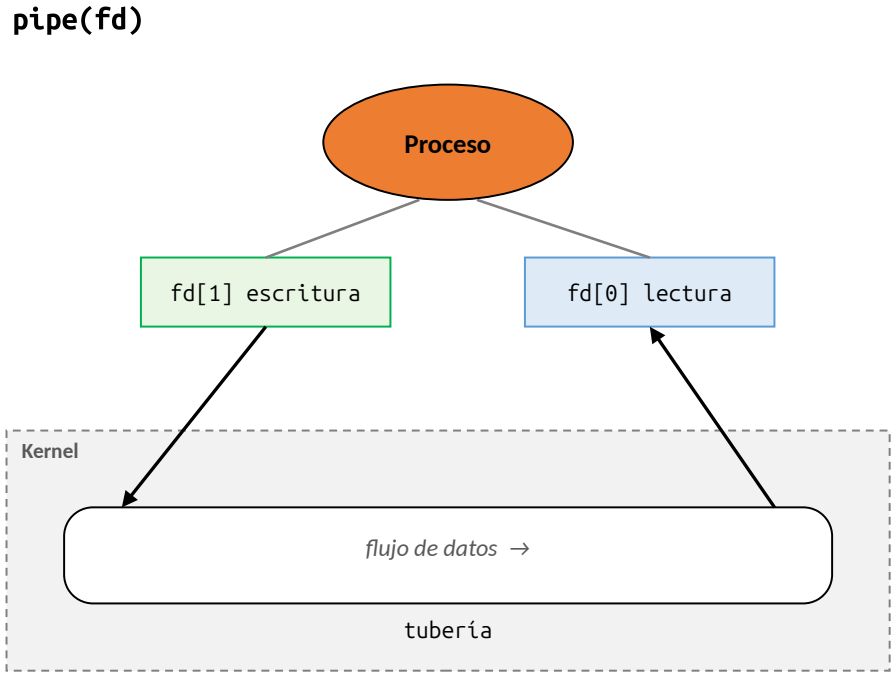

**Herencia de descriptores con fork()**




*Los cuatro descriptores apuntan a la misma tubería: hay que cerrar los que no se usan.*

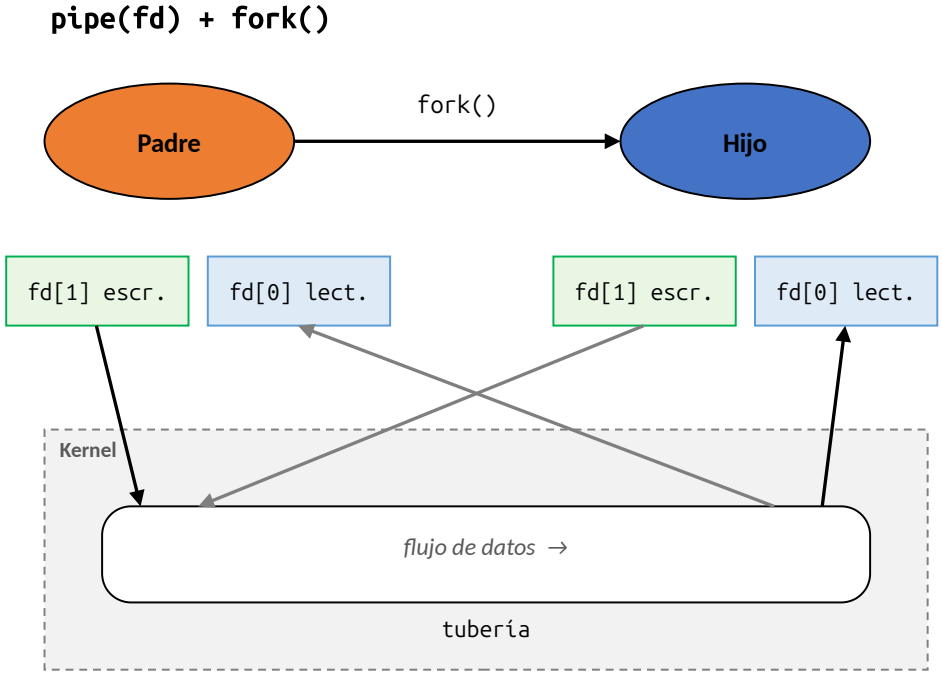
# Paso 1: crear la tubería con *pipe(fd)*

- `pipe(fd)` crea la tubería en el kernel y deja en el proceso **dos descriptores**: `fd[1]` para escribir y `fd[0]` para leer.
- Ambos extremos pertenecen al mismo proceso: puede escribir y leerse a sí mismo (ejemplo 1), pero eso no comunica nada.
- La utilidad aparece al combinarla con `fork()`: los descriptores se heredan.



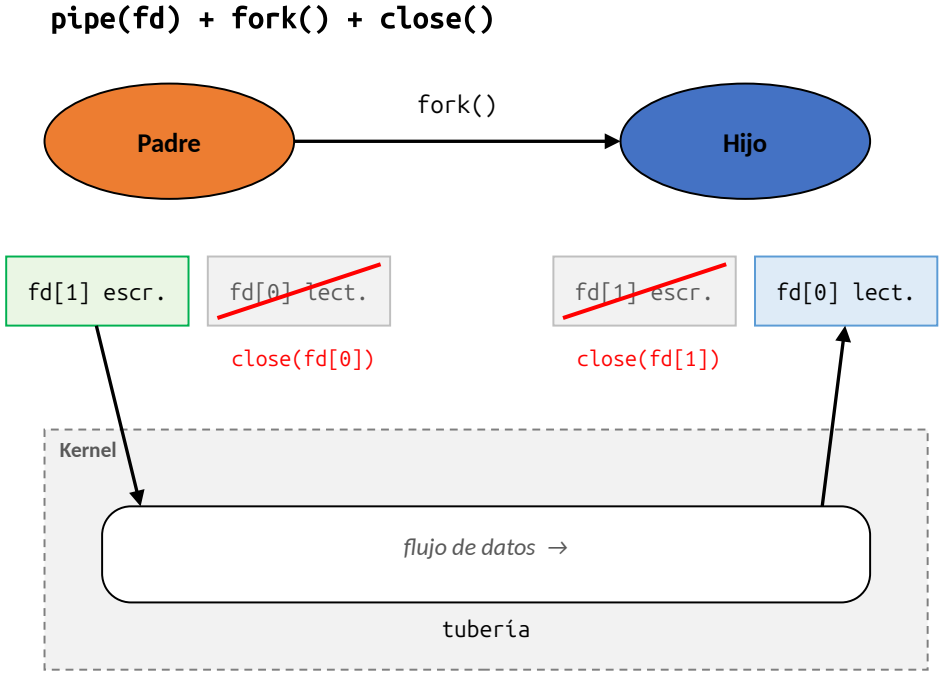
# Paso 2: *pipe(fd) + fork()*

- Tras `fork()` el hijo recibe copias de `fd[0]` y `fd[1]`: ahora **cuatro descriptores** apuntan a la misma tubería.
- Cualquiera de los dos procesos podría escribir o leer, lo que produce mezclas de datos y lecturas que nunca terminan.
- Por eso el siguiente paso es decidir el sentido de la comunicación y cerrar lo demás.



# Paso 3: cerrar los extremos que no se usan

- El padre escribirá: cierra `fd[0]`. El hijo leerá: cierra `fd[1]`.
- `read` devuelve 0 (fin de archivo) solo cuando **todos** los descriptores de escritura están cerrados: si el lector conserva `fd[1]`, nunca verá el final.
- Regla práctica: **una tubería, un sentido**. Para responder se abre otra tubería.



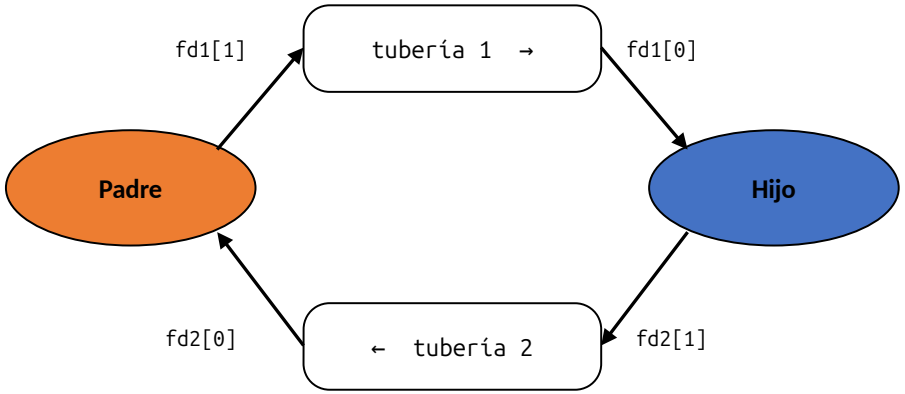
# Esqueleto de un programa con tubería

```
int fd[2];
pipe(fd);
child = fork();
if (!child) {          /* hijo: solo lee   */
    close(fd[1]);
    ... do_child() ...
    close(fd[0]);
} else {               /* padre: solo escribe */
    close(fd[0]);
    ... do_parent() ...
    close(fd[1]);
    wait(NULL);
}
```

- Crear la tubería **antes** de `fork()`: si se crea después, cada proceso tendría una tubería distinta.
- Cada rama cierra primero el extremo que no usa y, al terminar, el que sí usó.
- El padre espera al hijo con `wait()` para no dejar procesos zombis.
- Las funciones `do_child()` y `do_parent()` contienen las llamadas `read` y `write`.

**Error común:** olvidar cerrar `fd[1]` en el lector. El bucle de lectura se queda bloqueado para siempre porque el kernel cree que todavía hay un escritor.

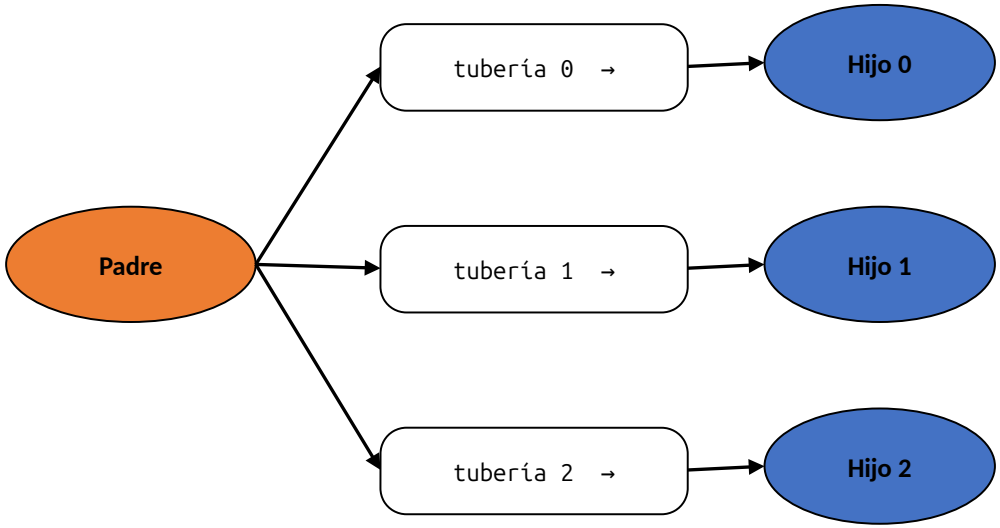
# Comunicación bidireccional: dos tuberías




```
int fd1[2], fd2[2];
pipe(fd1);  pipe(fd2);
child = fork();
if (!child) {                /* hijo */
    close(fd1[1]); close(fd2[0]);
    ... read(fd1[0]) ... write(fd2[1]) ...
    close(fd1[0]); close(fd2[1]);
} else {                     /* padre */
    close(fd1[0]); close(fd2[1]);
    ... write(fd1[1]) ... read(fd2[0]) ...
    close(fd1[1]); close(fd2[0]);
    wait(NULL);
}
```

- Cada tubería lleva datos en un solo sentido; para un diálogo entre padre e hijo se crean **dos** antes de `fork()`.
- El padre escribe en `fd1[1]` y lee de `fd2[0]`; el hijo lee de `fd1[0]` y escribe en `fd2[1]`. Cada uno cierra los otros dos extremos.
- Si ambos leen antes de escribir se produce un **interbloqueo**: hay que acordar el protocolo.

# Un padre y varios hijos: un arreglo de tuberías



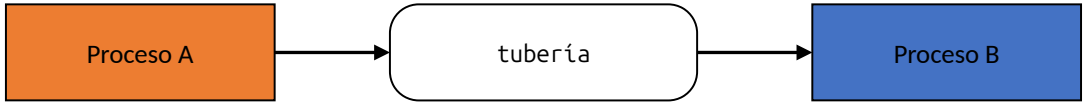

```
int fd[3][2];
for (int i = 0; i < 3; i++)
    pipe(fd[i]);
/* ... fork() por cada hijo ... */
for (int i = 0; i < 3; i++)
    write(fd[i][1], buf, strlen(buf));
```

- Un arreglo `fd[N][2]` guarda una tubería por hijo; el padre escribe en `fd[i][1]` y el hijo *i* lee de `fd[i][0]`.
- Cada hijo debe cerrar los extremos de las tuberías que no le corresponden.

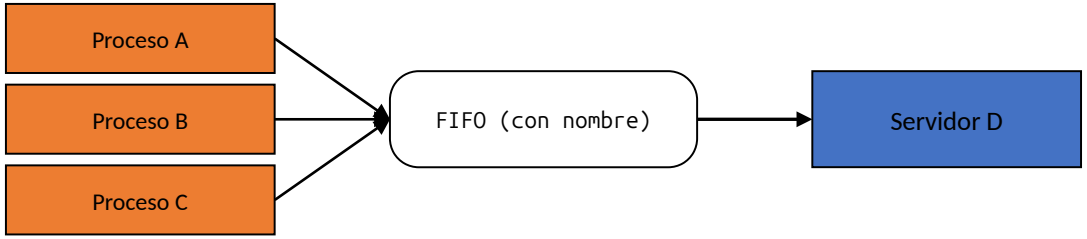
# Paradigmas de uso de las tuberías

Las tuberías sirven para implementar varios patrones de diseño clásicos:

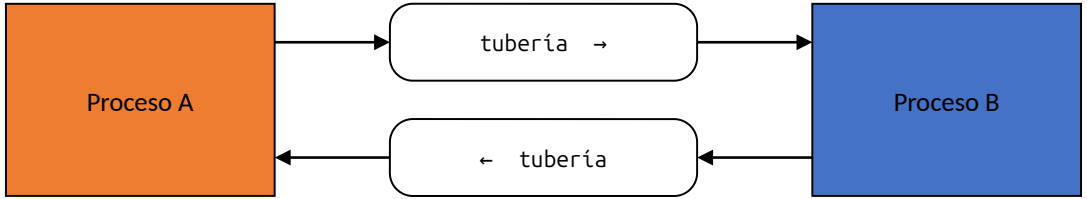
**Productor / consumidor**




**Cliente / servidor**




**Objeto activo**




*Una FIFO (tubería con nombre, mkfifo) permite que procesos no emparentados, como varios clientes, escriban a un mismo servidor.*

# Llamada al sistema *pipe*

***int pipe(int pipefd[2]);***

Crea una tubería **sin nombre** y semidúplex (unidireccional) entre procesos emparentados.

| | |
|---|---|
| **Include** | `<unistd.h>` |
| **Retorna** | **Éxito:** `0`, y `pipefd` queda cargado con los dos descriptores.<br>**Falla:** `-1` y `errno`: `EMFILE` (el proceso llegó a su límite de descriptores), `ENFILE` (el sistema llegó a su límite de archivos abiertos). |
| **Argumentos** | • ***int pipefd[2]***: arreglo de salida. `pipefd[0]` es el extremo de lectura y `pipefd[1]` el de escritura. |
| **Nota** | Los descriptores se usan con `read`, `write` y `close` como cualquier archivo; la tubería desaparece cuando se cierran todos. |

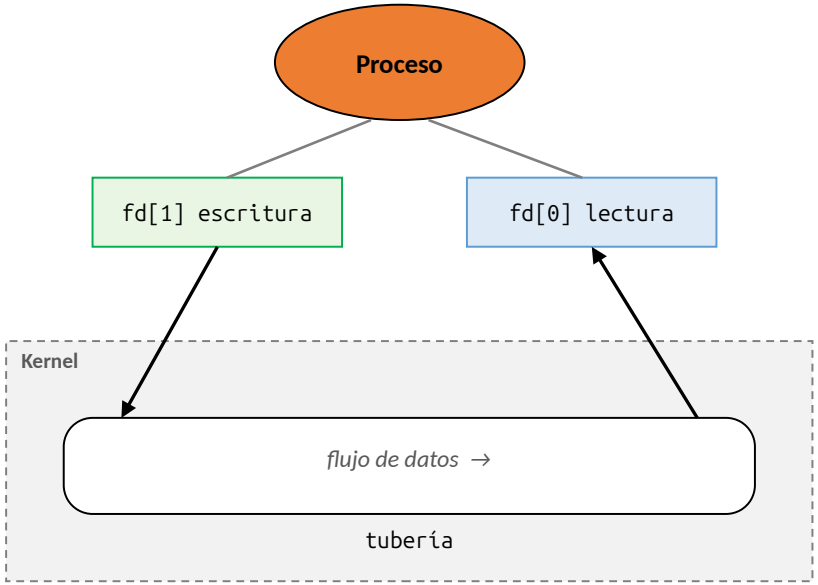
# Operar la tubería: *read*, *write* y *close*

| Llamada | Firma | Comportamiento con tuberías |
|---|---|---|
| `read` | *ssize_t read(int fd, void *buf, size_t n);* | Se bloquea si la tubería está vacía y queda algún escritor. Devuelve los bytes leídos, o 0 (fin de archivo) cuando todos los extremos de escritura están cerrados. |
| `write` | *ssize_t write(int fd, const void *buf, size_t n);* | Se bloquea si la tubería está llena. Si no queda ningún lector, el proceso recibe SIGPIPE (y write falla con EPIPE). Las escrituras de hasta PIPE_BUF bytes (4096 en Linux) son atómicas. |
| `close` | *int close(int fd);* | Libera el descriptor. La tubería y sus datos desaparecen cuando se cierran todos los descriptores en todos los procesos. |

Las tuberías transportan un **flujo de bytes**: no conservan los límites de cada `write`. Si el emisor escribe "hola" y "quit" seguidos, un solo `read` puede devolver "holaquit". Las colas de mensajes sí conservan los límites.

# Ejemplo 1: escribir y leer en el mismo proceso

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int main(void){
  int fd[2], n;
  char buffer_out[1024];
  char buffer_in[1024];
  char data[] = "Hola mundo pipes";
  if(pipe(fd) == -1){ perror("pipe"); return EXIT_FAILURE; }
  strcpy(buffer_out, data);
  printf("[%d]write:--> %s\n", getpid(), buffer_out);
  if(write(fd[1], buffer_out, strlen(buffer_out)) == -1)
    perror("write");
  n = read(fd[0], buffer_in, sizeof(buffer_in) - 1);
  buffer_in[n] = '\0';
  printf("[%d]read:<-- %s\n", getpid(), buffer_in);
  return EXIT_SUCCESS;
}
```

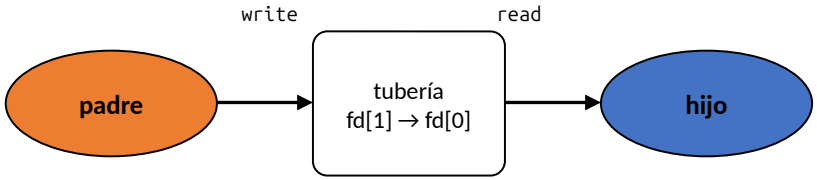



```
[7712]write:--> Hola mundo pipes
[7712]read:<-- Hola mundo pipes
```

Muestra que los datos pasan por el kernel y vuelven, pero un solo proceso no necesita una tubería: la comunicación real empieza con `fork()`.

# Ejemplo 2: el padre envía un mensaje al hijo

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
int main(void){
  int fd[2], n;
  char buff[50];
  if(pipe(fd) == -1){ perror("pipe"); return EXIT_FAILURE; }
  if(fork()){
    char msg[] = "Mensaje de texto";
    close(fd[0]);
    if(write(fd[1], msg, strlen(msg)) == -1)
      perror("write");
    printf("[%d]write:--> %s\n", getpid(), msg);
    wait(NULL);
  }
  else{
    close(fd[1]);
    n = read(fd[0], buff, sizeof(buff) - 1);
    buff[n] = '\0';
    printf("[%d]read:<-- %s\n", getpid(), buff);
  }
  return EXIT_SUCCESS;
}
```

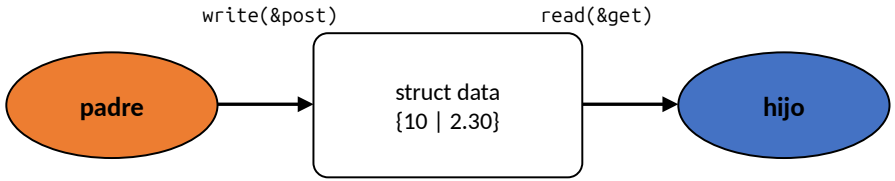



```
[7712]write:--> Mensaje de texto
[7713]read:<-- Mensaje de texto
```

El padre cierra `fd[0]` y el hijo `fd[1]` antes de usar la tubería. El hijo se bloquea en `read` hasta que llega el mensaje: la sincronización la aporta el kernel.

# Ejemplo 3: enviar una estructura por la tubería

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
struct data{
  int a;
  float b;
};
int main(void){
  int fd[2], n;
  if(pipe(fd) == -1){ perror("pipe"); return EXIT_FAILURE; }
  if(fork() == 0){
    struct data get;
    close(fd[1]);
    n = read(fd[0], &get, sizeof(struct data));
    printf("[%d]read:<-- [%d|%.2f] (%d bytes)\n",
           getpid(), get.a, get.b, n);
  }
  else{
    struct data post;
    close(fd[0]);
    post.a = 10;
    post.b = 2.3;
    printf("[%d]write:--> [%d|%.2f]\n",
           getpid(), post.a, post.b);
    if(write(fd[1], &post, sizeof(struct data)) == -1)
      perror("write");
    wait(NULL);
  }
  return EXIT_SUCCESS;
}
```

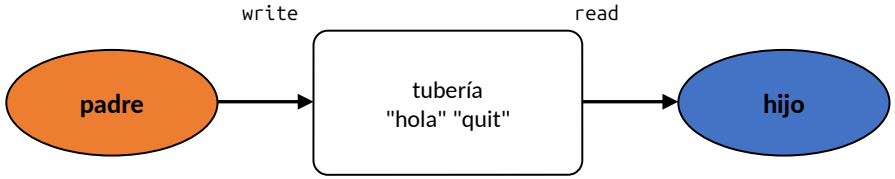



```
[7712]write:--> [10|2.30]
[7713]read:<-- [10|2.30] (8 bytes)
```

Se envían los `sizeof(struct data)` bytes en binario: no hay conversión a texto. Emisor y receptor deben usar la misma definición de la estructura (y la misma arquitectura).

# Ejemplo 4: mensajes de texto hasta "quit"

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#define MAX_READ 256
char buff[MAX_READ];
int main(void){
  int fd[2], n;
  if(pipe(fd) == -1){ perror("pipe"); return EXIT_FAILURE; }
  if(fork() != 0){
    close(fd[0]);
    printf("write quit to exit\n");
    do{
      if(fgets(buff, MAX_READ, stdin) == NULL) break;
      if(strlen(buff) > 1){
        buff[strlen(buff)-1] = '\0';
        printf("[%d]write-->:%s\n", getpid(), buff);
        if(write(fd[1], buff, strlen(buff)) == -1)
          perror("write");
      }
    }while(strcmp(buff, "quit") != 0);
    close(fd[1]);
    wait(NULL);
  }
  else{
    close(fd[1]);
    while((n = read(fd[0], buff, MAX_READ - 1)) > 0){
      buff[n] = '\0';
      printf("[%d]read<--:%s\n", getpid(), buff);
    }
  }
  return EXIT_SUCCESS;
}
```




El hijo termina cuando `read` devuelve 0, es decir, cuando el padre cierra `fd[1]`. Si el padre escribe "hola" y "quit" antes de que el hijo lea, un solo `read` recibe "holaquit": la tubería no separa mensajes.

# Resumen

- Una tubería es un canal **unidireccional** de bytes que vive en el kernel y se usa con `read` y `write`.
- `pipe(fd)` crea dos descriptores: `fd[0]` lectura y `fd[1]` escritura; `fork()` los hereda.
- Cada proceso cierra el extremo que no usa: así `read` detecta el fin de archivo y no quedan procesos bloqueados.
- Para diálogo hacen falta dos tuberías; para varios hijos, un arreglo de tuberías; para procesos no emparentados, una FIFO con nombre.
- No hay límites de mensaje ni tipos: para eso están las colas de mensajes; para compartir datos sin copias, la memoria compartida.

# Referencias y lecturas


- Kerrisk, M. *The Linux Programming Interface*, cap. 44: Pipes and FIFOs. No Starch Press, 2010.
- Stevens, W. R. *UNIX Network Programming*, *vol. 2: Interprocess Communications*, cap. 4. Prentice Hall, 1999.
- Carretero, J. et al. *Sistemas operativos: una visión aplicada*, servicios POSIX de comunicación. McGraw-Hill, 2007.
- Páginas de manual: `man 2 pipe`, `man 7 pipe`, `man 3 mkfifo`.
- Código de los ejemplos: `example_codes/03-pipes/` en el repositorio del curso.